Reimplementing ConvNeXt for CIFAR-10 Image Classification

Algorithm Search, Implementation and Evaluation with AI Assistance

CISC3024 Pattern Recognition — AI Assignment #2

Student: Chao Wen Jie (UC325315)
Date: 10 October 2026

Table of Contents

Table of Contents 2
1. How I Asked AI Tools to Find the Algorithm 3
1.1 Step 1 — turning the assignment into checkable constraints 3
1.2 Step 2 — divergent search for candidates 3
1.3 Step 3 — converging on ConvNeXt 4
1.4 Step 4 — verifying the provenance of the algorithm 4
1.5 How the prompts were structured 5
2. Algorithm Description 6
2.1 Motivation and central idea 6
2.2 The modernisation roadmap 6
2.3 The ConvNeXt block 6
2.4 Stage layout, downsampling and classifier head 7
2.5 Adaptation to 32x32 inputs (the only structural change) 7
2.6 Training recipe and regularisation 7
3. How the AI Implemented the Algorithm 9
3.1 Working protocol 9
3.2 Generated project structure 9
3.3 Engineering decisions worth highlighting 9
3.4 Verification performed on the generated code 10
3.5 Three engineering problems that only appeared on real hardware 10
3.6 Reproducibility measures 11
4. Experiment Settings and Results 12
4.1 Data 12
4.2 Settings of the main experiment 12
4.3 Main results 12
4.4 Training behaviour 13
4.5 Per-class results and error analysis 13
4.6 Controlled comparison: the effect of numerical precision 15
4.7 CPU baseline for pipeline validation 16
4.8 Discussion and limitations 16
5. What I Have Learnt from This AI Assignment 17
5.1 An AI agent can own the entire pipeline, but verification remains the human contribution 17
5.2 Numerical precision is an engineering problem, not a detail 17
5.3 Platform behaviour belongs to the experiment 17
5.4 Reproducibility is a design decision taken in advance 17
5.5 How to prompt an AI agent effectively 17
5.6 What I would do differently next time 18
6. Source Code Webpage Link 19
6.1 What the repository contains 19
6.2 How to run the code 19
References 20

1. How I Asked AI Tools to Find the Algorithm

Three constraints are fixed before any modelling decision: the algorithm has to come from the Deep CNN or Deep Autoencoder family, it has to have been published between 2022 and 2026, and it has to be applied to a computer-vision or pattern-recognition task. With those in place, the AI tools were not given an open question such as “which CNN is good?”. The first task was to turn the assignment text into a checkable list of constraints, and only after that came the search for candidates that satisfy all of them.

1.1 Step 1 — turning the assignment into checkable constraints

The first prompt asked the AI to pull the mandatory requirements out of the assignment word for word and restate them as a list that could be checked off one by one. Having that list up front made it easy to rule out tempting but non-compliant options early on, such as pure vision transformers, or repositories that only ship pre-trained weights with no paper behind them.

Table 1. Constraints extracted from the assignment and used as search filters.

#	Constraint	How it was checked
C1	Category must be Deep CNN or Deep Autoencoder	Model is built only from standard convolution / linear / normalisation operators — no self-attention anywhere in the code
C2	Proposed in 2022–2026 with a verifiable source	Formal publication (CVPR 2022) plus arXiv identifier and open-access link
C3	Applied to CV / pattern recognition	Image classification on CIFAR-10
C4	All code must be generated by AI	Every source file is embedded in the AI-generated project and in the generated Colab notebook; dialogue records kept
C5	Report must be in English, Word or PDF	This document
C6	Report must cover the six required items	Sections 1–6 of this report
C7	An accessible web link to the source code	Section 6 (link to be finalised by the author before submission)

1.2 Step 2 — divergent search for candidates

The second prompt asked for a shortlist of Deep-CNN and autoencoder algorithms from 2022–2026, each with its paper title, venue, a one-sentence summary of the contribution and, most importantly for this project, an honest assessment of how well it would run on limited hardware (a CPU-only laptop and a free-tier GPU). Asking for that feasibility column from the start kept out algorithms whose cost only makes sense on a large cluster.

Table 2. Candidate pool returned by the AI and the criteria used for the decision.

Candidate	Year	Category	Feasibility on limited hardware	Decision
ConvNeXt (A ConvNet for the 2020s)	2022	Deep CNN	High — pure PyTorch, no custom CUDA kernels; module boundaries are clean and each design element can be verified independently	Selected
ConvNeXt V2 (FCMAE)	2023	Deep CNN + self-supervised pretraining	Medium — an extra masked-autoencoder pretraining stage roughly doubles the training cost	Rejected (cost)

Candidate	Year	Category	Feasibility on limited hardware	Decision
Masked Autoencoders (MAE)	2022	Deep Autoencoder	Low to medium — compliant with the category rule but the pretraining budget is high for a first deliverable	Rejected (cost)

1.3 Step 3 — converging on ConvNeXt

The third prompt asked the AI to rank the candidates on three axes — how well each one fits the required category, how clearly it can be implemented and how affordable it is to train — and to state what adapting it to a small-resolution dataset would cost. The reasoning behind the final choice was as follows.

- Category compliance. ConvNeXt contains no attention operator at all. It is a pure convolutional network, which is the plainest reading of the “Deep CNN” constraint.
- Implementation clarity. The architecture splits cleanly into a stem, four stages and a classifier head, and each design element — the inverted bottleneck, the 7x7 depthwise convolution, LayerNorm in place of BatchNorm, the separate downsampling layers, layer scale — can be implemented and verified on its own, which is what Section 2 needs.
- Compute affordability. The project defines five presets (atto, femto, pico, nano and tiny). The smallest is light enough to run the whole pipeline end to end on a CPU in about 19 minutes, while the tiny preset (27.8 M parameters) matches the paper’s ConvNeXt-T and trains on a single 8 GB laptop GPU.
- Small-resolution adaptation. The paper’s stem is a 4x4 convolution with stride 4, which suits 224x224 ImageNet images. On 32x32 CIFAR-10 images the same stem would shrink the feature maps to 8 → 4 → 2 → 1 and throw away most of the spatial detail. The AI therefore suggested replacing it with a 3x3 stride-1 convolution, giving feature-map sizes of 32 → 16 → 8 → 4 while leaving every other design element of the paper untouched (Section 2.5).

1.4 Step 4 — verifying the provenance of the algorithm

Since the assignment requires a verifiable source, a separate prompt asked for the full bibliographic record, the licence of the official implementation and an explicit statement of how ConvNeXt satisfies the category and year constraints.

Table 3. Verifiable provenance of the selected algorithm.

Item	Value
Paper	A ConvNet for the 2020s — Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, Saining Xie
Venue / year	CVPR 2022, pp. 11976–11986 (IEEE/CVF Conference on Computer Vision and Pattern Recognition)
Preprint	arXiv:2201.03545 (first released 10 January 2022), https://arxiv.org/abs/2201.03545
Open access	https://openaccess.thecvf.com/content/CVPR2022/html/Liu_A_ConvNet_for_the_2020s_CVPR_2022_paper.html
Official code	https://github.com/facebookresearch/ConvNeXt (MIT License)
Category check	Built exclusively from convolutions, linears and normalisation layers → Deep CNN (satisfies C1)

Item	Value
Year check	Published 2022, inside the required 2022–2026 window (satisfies C2)

1.5 How the prompts were structured

Every search round followed the same three-part pattern, which is why the dialogue records could later be quoted directly in this report: ask a constrained question; ask the AI to justify its answer with evidence that can be checked; and ask for the result to be saved to a file in the project archive. The five rounds are archived as the algorithm-search dialogue, the candidate list, the algorithm profile and the iteration summary. Two habits mattered most: getting the AI to argue against its own proposal, which is how the 4x4 stride-4 stem problem came to light, and insisting that every claim about the literature come with a link or identifier that can be opened and verified.

2. Algorithm Description

This section describes ConvNeXt as it was published, then explains how it was adapted to 32x32 inputs and which training recipe was used.

2.1 Motivation and central idea

After 2020, vision transformers took over from convolutional networks as the dominant backbone, and most of the credit went to self-attention. ConvNeXt tests the opposite idea: that the gap between transformers and ConvNets is due mainly to training techniques and architectural choices that ConvNets had simply not adopted, rather than to attention itself. The authors start from a standard ResNet-50 and modernise it one step at a time, borrowing design elements from transformers (Swin Transformer in particular) and measuring the effect of each change. What comes out is a family of pure convolutional networks whose accuracy matches comparable Swin transformers, and is slightly better on detection and segmentation.

2.2 The modernisation roadmap

The roadmap is the paper's main contribution: eight design decisions, each added on its own and each measured in ImageNet top-1 accuracy (the ResNet-50 baseline is 76.1 %).

Table 4. Design elements of the roadmap and what ConvNeXt adopts.

#	Design element	ResNet-50 baseline	ConvNeXt choice	Rationale
1	Macro design (stage ratio)	(3, 4, 6, 3)	(3, 3, 9, 3)	Match the 1:1:3:1 stage ratio of Swin-T; more capacity in the third stage
2	Stem	7x7 conv, stride 2 + max-pool	4x4 conv, stride 4 (patchify)	Non-overlapping patches, as in ViT (adapted to 3x3 stride 1 for CIFAR-10, Section 2.5)
3	ResNeXt-ify	3x3 conv, width 64	Grouped / depthwise convolution	Depthwise convolution with a larger width achieves comparable accuracy at lower cost
4	Inverted bottleneck	Bottleneck: wide → narrow → wide	1x1 expand x4 → depthwise → 1x1 project	Follows the transformer block, where the MLP is four times wider than the input
5	Large kernel	3x3 convolutions	7x7 depthwise convolution	Enlarges the effective receptive field, analogous to self-attention's global mixing
6	Micro design — activation	ReLU	GELU, and only one activation per block	Matches the transformer MLP; fewer activations reduce latency and improve accuracy
7	Micro design — normalisation	BatchNorm	LayerNorm	BatchNorm is unnecessary for training stability in the modernised design and harms transfer
8	Downsampling	stride-2 in the first block of each stage	Separate 2x2 stride-2 convolution between stages	Decouples spatial reduction from the residual path

2.3 The ConvNeXt block

Each stage is a stack of identical blocks. In the final design a block consists of a 7x7 depthwise convolution, a LayerNorm, a 1x1 convolution that widens the channel dimension fourfold, a GELU activation, a 1x1 convolution that projects back to the original width, a learnable per-channel rescaling (layer scale), a stochastic-depth (drop-path) regulariser and a residual connection. Writing the input as x, with channel dimension C, an expansion factor of 4 and a layer-scale vector $\gamma \in \mathbb{R}^C$, the block computes:

$$y = Linear_2(GELU(Linear_1(LayerNorm(DWConv_{7\times7}(x))))) \quad (1)$$

$$z = \gamma \cdot y \quad (2)$$

$$out = x + DropPath(z) \quad (3)$$

Two details matter in practice. First, the depthwise convolution is the only operator with a large receptive field, which is why the 7x7 kernel, rather than 3x3, is essential. Second, equation (2) is the layer scale: $\gamma \in \mathbb{R}^C$ is a learnable per-channel vector, initialised to 10^{-6}, that multiplies the residual branch, so that early in training every block behaves almost as an identity mapping. This is what keeps deep networks with few normalisation layers stable to optimise. The second point returns in Section 4.6, where it turns out to interact badly with 16-bit floating point.

2.4 Stage layout, downsampling and classifier head

The network has four stages. Between stage i and stage i+1, a dedicated downsampling layer applies a LayerNorm followed by a 2x2 stride-2 convolution. The final feature map is normalised with a single-group GroupNorm (equivalent to a per-sample, channel-first LayerNorm), globally average-pooled and classified by one linear layer. The five presets used in this project, of which tiny is the paper's ConvNeXt-T, are listed below.

Table 5. Model presets defined in the project (parameter counts measured on the implemented model).

Preset	Depths	Dims	Parameters	Role in this project
atto	(2, 2, 6, 2)	(40, 80, 160, 320)	3.38 M	CPU end-to-end MVP run
femto	(2, 2, 6, 2)	(48, 96, 192, 384)	5.2 M	Reserved for ablations
pico	(2, 2, 6, 2)	(64, 128, 256, 512)	9.0 M	Fast GPU experiments
nano	(2, 2, 8, 2)	(80, 160, 320, 640)	15.6 M	Reserved for ablations
tiny	(3, 3, 9, 3)	(96, 192, 384, 768)	27.83 M	Main experiment (paper's ConvNeXt-T)

2.5 Adaptation to 32x32 inputs (the only structural change)

ImageNet images are 224x224, so the paper's 4x4 stride-4 stem is a good fit there: the feature maps run 56 → 28 → 14 → 7. CIFAR-10 images are only 32x32, and the same stem would give 8 → 4 → 2 → 1, leaving 1x1 maps in the last stage and discarding most of the spatial structure. The implementation therefore uses a 3x3 stride-1 convolution as the stem, which gives 32 → 16 → 8 → 4 across the four stages. Everything else in the paper is kept as published: the (3, 3, 9, 3) stage ratio, the 4x inverted bottleneck, the 7x7 depthwise convolution, GELU, LayerNorm, the separate downsampling layers, layer scale and stochastic depth. The stem can be switched at run time (--stem-mode conv|patchify), so the original configuration is still available.

2.6 Training recipe and regularisation

The recipe follows the paper’s training practice, adjusted for a small dataset. It uses AdamW with a base learning rate of 1e-3 and weight decay 0.05, with one-dimensional parameters (normalisation scales and offsets, and all biases) exempt from the decay; a five-epoch linear warm-up followed by cosine decay to 1e-5; label smoothing of 0.1; Mixup and CutMix on 50 % of the batches, with alpha values of 0.8 and 1.0; augmentation suited to 32x32 images rather than RandomResizedCrop, namely a random crop with 4-pixel reflect padding, a horizontal flip and random erasing with probability 0.25; stochastic depth with a maximum rate of 0.1 that grows linearly with block index; and layer scale initialised to 1e-6.

3. How the AI Implemented the Algorithm

The implementation was produced by an AI coding agent under a simple arrangement: the agent writes all of the code, and the human only states the requirements, runs the commands it produces and inspects the output. Nothing in the project is hand-written. This section describes what was generated, the engineering decisions behind it and the checks used to decide whether the code could be trusted.

3.1 Working protocol

1. Describe the deliverable and its acceptance criteria in plain language, including the hardware constraints.
2. Let the AI generate the complete project (not a single script), with a documented command-line interface and deterministic output paths.
3. Run the generated commands; keep the raw logs, metrics and figures as evidence.
4. Ask the AI to analyse the produced artefacts and to justify any follow-up change with measurements rather than opinion.
5. Record the dialogue, so that every change remains traceable to a request.

3.2 Generated project structure

Table 6. Structure of the generated code base (24 Python files plus a self-contained Colab notebook).

Name	**Responsibility**
configs.py	Single source of truth for data / model / training hyper-parameters, dataclass-based, fully overridable from the command line, serialised to JSON for reproducibility
train.py, evaluate.py	Entry points: train-validation-test pipeline with artefact export; standalone evaluation of a saved checkpoint (with optional test-time augmentation)
convnext.py	ConvNeXt: layer-scale block, depthwise convolution, LayerNorm (both layouts), separate downsampling, five presets, parameter summary
trainer.py	Training loop: AdamW with parameter grouping, warm-up + cosine schedule, gradient accumulation, gradient clipping, AMP with an automatic safety net, checkpointing, resume, per-epoch metric export
evaluator.py	Top-1 / Top-5 accuracy, per-class accuracy, confusion matrix, test-time augmentation with identical protocol for all artefacts
data/	CIFAR-10 loading with a single base dataset per split, deterministic split protocol, synthetic dataset for zero-download smoke tests
utils/	Device selection and memory-adaptive batch-size probe, AMP compatibility layer, mixup / cutmix, CSV export, logging, seeding, visualisation, TTA
notebooks/	Generator that writes a self-contained Colab notebook from the sources, plus a verbatim self-check that fails if notebook and sources ever diverge
fetch_cifar10.py	Mirror probing, multi-connection chunked download, MD5 verification and extraction for the 170 MB CIFAR-10 archive

3.3 Engineering decisions worth highlighting

- One base dataset per split. torchvision.datasets.CIFAR10 re-checks the MD5 checksums of the 186 MB archive every time it is constructed. Building the dataset once per split and

layering the different augmentation pipelines on top through a TransformSubset wrapper cut the hot path from 4.5 s to 1.16 s without changing the index semantics.

- Memory-adaptive batch size. Before training starts, the code runs a real forward, backward and optimiser step on the target device and adjusts the batch size until it fits, logging the measured peak. When the batch size has to be reduced, gradient accumulation makes up the difference so the effective batch stays close to the requested one. Section 4.6 describes the extra safety margin that Windows turned out to require.
- A compatibility and safety layer for mixed precision. The project supports both the pre-2.4 and the current PyTorch AMP APIs, prefers bfloat16 wherever the device supports it natively, records the gradient scaler's dynamic loss scale every epoch and can fall back to full precision on its own.
- Experiment isolation. Each run is identified by a tag (--run-tag), and all artefacts go to a per-run directory, so runs that are repeated or run in parallel cannot overwrite or cross-reference one another.
- Incremental persistence. The training history is written to disk after every epoch, and both the best and the last checkpoint hold the model, optimiser, scheduler and scaler state, the history and the full configuration. That is what made the mid-training interruption in Section 4.4 recoverable without losing a single epoch.

3.4 Verification performed on the generated code

Table 7. Checks applied to the AI-generated implementation.

Check	How it was performed	Result
Smoke test without any download	Train the smallest preset for 2 epochs on locally generated synthetic data	Pipeline runs end to end in ~20 s
End-to-end run on real data (CPU)	atto preset, 4,500 training images, 10 epochs	Test Top-1 45.05 % — clearly above the 10 % chance level, so the pipeline genuinely learns
Determinism of the split protocol	Re-computed the training / validation / test index sets and compared them with the archived split index files	Identical index sets
Notebook-versus-source consistency	The notebook generator re-reads the sources and compares them verbatim with the embedded copies	Passes (24/24 files); the check exits with a non-zero code otherwise
Numerical-health monitoring	Every epoch logs the gradient-scaler loss scale; checkpoint files contain optimiser and scaler states that can be inspected afterwards	This is what exposed the failure analysed in Section 4.6

3.5 Three engineering problems that only appeared on real hardware

The pipeline was validated on a CPU first and then moved to a GPU, where three problems surfaced that reading the code had not revealed. All three were diagnosed from the artefacts of the runs themselves and then fixed, and they are described here because documenting this kind of work is part of the assignment.

Table 8. Problems found by running the pipeline, and the fixes applied to the generated code.

Problem	Symptom	Diagnosis	Fix
16-bit overflow of the loss scale (T4 GPU, fp16)	60 epochs completed without error, but the loss never moved and the test accuracy stayed at 25.73 %	The checkpoint contained scaler_state.scale = 0.0156 (default 65536) and Adam's second-moment buffers of the early blocks were exactly zero, i.e. those weights had never been updated	Prefer bfloat16 (same exponent range as fp32); log the loss scale every epoch; abort to fp32 automatically if the scale collapses
Over-committed GPU memory (Windows/WDDM)	The batch-size probe reported a peak of 10.8 GB on an 8 GB card, after which the real training failed immediately with an out-of-memory or a CUDA driver error	Windows allows CUDA to allocate beyond physical video memory, so the probe can “pass” a batch size that does not really fit	Require the probe peak to stay below 70 % of the free memory, reset peak statistics before every probe, and treat driver-side memory errors as “does not fit” (batch 512 → 128 on the 8 GB laptop)
Commit-charge exhaustion in the multi-process data loader (Windows)	The process died after about two hours with Couldn't open shared file mapping ... error code: 1455	PyTorch's multi-process loader moves every batch through a shared file mapping; on Windows this slowly consumes system commit charge	Run long jobs with a single-process loader, and resume from the last checkpoint when a run is interrupted

3.6 Reproducibility measures

- Fixed seeds for Python, NumPy, PyTorch and the loader workers; the training / validation split is derived from a fixed-seed permutation and archived as index files.
- Every run writes its complete hyper-parameter set to config.json, which is also embedded in the checkpoints.
- Per-epoch metrics are exported both as JSON and as a CSV with a fixed column order, so results of different experiments can be stacked into a single comparison table.
- The Colab notebook is generated from the sources and then self-checked, so the notebook and the repository cannot drift apart.

4. Experiment Settings and Results

Three runs are reported here: a CPU end-to-end MVP that validates the pipeline, the main full-scale GPU run, and a failed fp16 run that serves as a controlled comparison of the effect of numerical precision.

4.1 Data

CIFAR-10 contains 60,000 32x32 RGB images in ten classes (6,000 each). The official split was followed exactly: the 50,000 training images were shuffled with a fixed seed (42) and divided into 45,000 for training and 5,000 for validation, and the official 10,000-image test set was never used for training or model selection. Augmentation was applied to the training split only; validation and test images went through a deterministic transform, namely tensor conversion and normalisation with the CIFAR-10 channel statistics (0.4914, 0.4822, 0.4465) and (0.2470, 0.2435, 0.2616).

4.2 Settings of the main experiment

Table 9. Configuration of the full-scale run (full_convnext_tiny_cifar10); the same values are archived in config.json.

Item	Value
Model	ConvNeXt-Tiny — depths (3, 3, 9, 3), widths (96, 192, 384, 768), 27,825,802 parameters
Stem	3x3 convolution, stride 1 (32x32 adaptation); stage resolutions 32 → 16 → 8 → 4
Input / classes	32x32 RGB / 10
Data split	45,000 train / 5,000 validation / 10,000 test (official test set held out)
Optimiser	AdamW, learning rate 1e-3, weight decay 0.05, one-dimensional parameters exempt from decay, betas (0.9, 0.999)
Schedule	5 epochs linear warm-up, then cosine decay to 1e-5, 60 epochs total
Batch size	128 (effective batch 128, no gradient accumulation)
Augmentation	Random crop 32 with 4-pixel reflect padding; horizontal flip; random erasing p = 0.25; Mixup / CutMix on 50 % of batches (alpha 0.8 / 1.0)
Regularisation	Label smoothing 0.1; stochastic depth 0 → 0.1 over depth; layer-scale initialised at 1e-6; gradient clipping at norm 1.0
Mixed precision	bfloat16 autocast with gradient scaling; minimum observed loss scale 2.10e6; no fallback to fp32 was triggered
Hardware / software	NVIDIA RTX 4060 Laptop GPU (8 GB), PyTorch 2.14.1+cu130, CUDA 13.0, Python 3.14; data loading with a single process (see Section 3.5)
Seed / wall-clock time	42 / 1 h 55 min for the resumed segment (about 3 h 05 min including the interrupted segment)

4.3 Main results

The checkpoint with the best validation accuracy (epoch 59, 93.50 %) was evaluated on the held-out test set twice: once with a single view and once with test-time augmentation, which averages the identity and horizontally flipped views.

Table 10. Test-set results of the full-scale run.

Metric	Single view	With test-time augmentation
Test cross-entropy loss (label smoothing 0.1)	0.6842	0.6842 (loss is computed without TTA)
Top-1 accuracy	92.66 %	92.87 %
Top-5 accuracy	99.56 %	99.61 %
Macro-average accuracy	92.67 %	92.87 %
Correct / total	9,266 / 10,000	9,287 / 10,000
Best validation accuracy (model selection)	93.50 % (epoch 59)	—

For reference, random guessing across ten classes gives 10 % Top-1 and 50 % Top-5, so the trained model is 82.9 and 49.6 percentage points above chance. Test-time augmentation adds 0.21 percentage points, within the +0.2 to +0.5 range that the project's design notes anticipated for this technique.

4.4 Training behaviour

Training behaved as expected in two phases: a fast phase in which validation accuracy climbed from 43.0 % at epoch 1 to 90.7 % at epoch 30, followed by a slow refinement that reached 93.50 % at epoch 59. The validation loss fell steadily to 0.6697 and never drifted away from the training loss, so no overfitting occurred; that is the effect of Mixup/CutMix, label smoothing, stochastic depth and strong augmentation. The learning rate followed the warm-up and cosine schedule exactly, from 1e-3 down to 1.08e-5.

One operational incident is worth recording, because it shows up clearly in the logs. After epoch 32 the process was killed by a Windows-specific memory error in the multi-process data loader (Section 3.5, third row). The run was resumed from the last checkpoint with --resume auto, which restored the model, the optimiser, the learning-rate schedule and the full history. Training continued from epoch 33 and the accuracy curve crossed the break without a visible step (90.70 % → 91.44 %). The only trace left behind is the per-epoch time rising from about 120 s to about 230 s, a consequence of changing the number of data-loader workers. No epoch was lost.

4.5 Per-class results and error analysis

Table 11. Per-class test accuracy (1,000 images per class).

Class	Top-1 (TTA)	Top-1 (single view)	Class	Top-1 (TTA)	Top-1 (single view)
airplane	93.80 %	93.50 %	dog	88.20 %	87.90 %
automobile	97.90 %	98.20 %	frog	96.20 %	95.90 %
bird	90.90 %	90.20 %	horse	94.20 %	94.60 %
cat	82.80 %	82.10 %	ship	96.50 %	96.30 %
deer	93.40 %	92.90 %	truck	94.80 %	95.10 %

The confusion matrix has a clear structure. The only pair confused in both directions by more than 8 % is cat ↔ dog (8.2 % of cats predicted as dogs and 8.1 % of dogs as cats), the classic fine-grained difficulty of similar fur texture and pose. Beyond that pair, the largest errors are truck → automobile (2.9 %), airplane → ship (2.2 %), bird → frog (2.2 %) and deer → frog (2.0 %). The most accurate classes — automobile, ship, frog and truck — are the ones with distinctive overall

shape and background, which supports the reading that the remaining errors sit in shape- and texture-similar quadruped and vehicle pairs rather than being scattered at random.

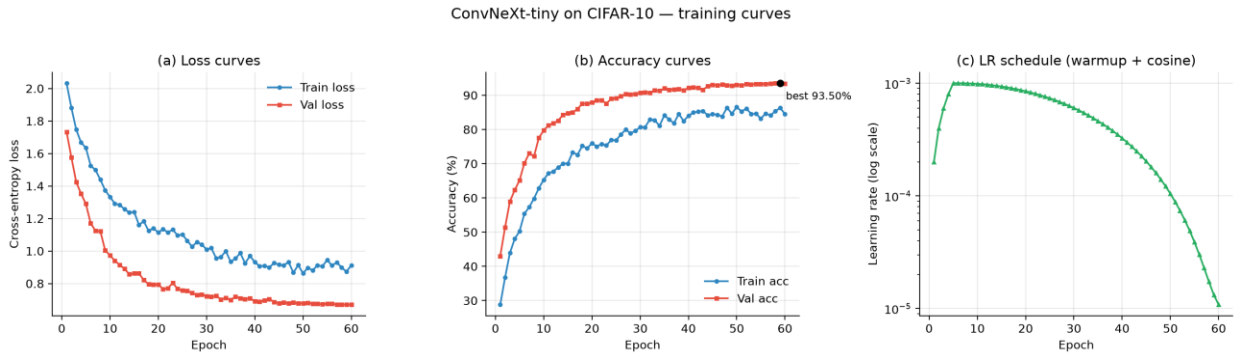


Figure 1. Training curves: (a) cross-entropy loss, (b) top-1 accuracy with the best epoch marked, (c) the warm-up plus cosine learning-rate schedule.

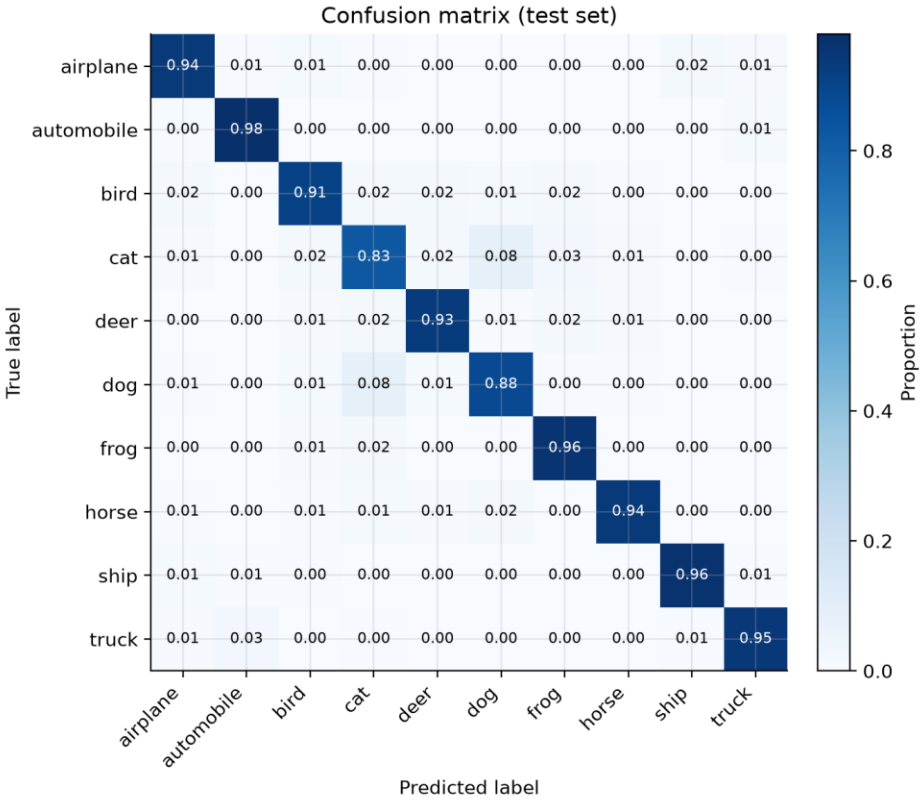


Figure 2. Row-normalised confusion matrix on the test set (TTA protocol). The diagonal gives the per-class recall; the dominant off-diagonal mass is the cat/dog pair.

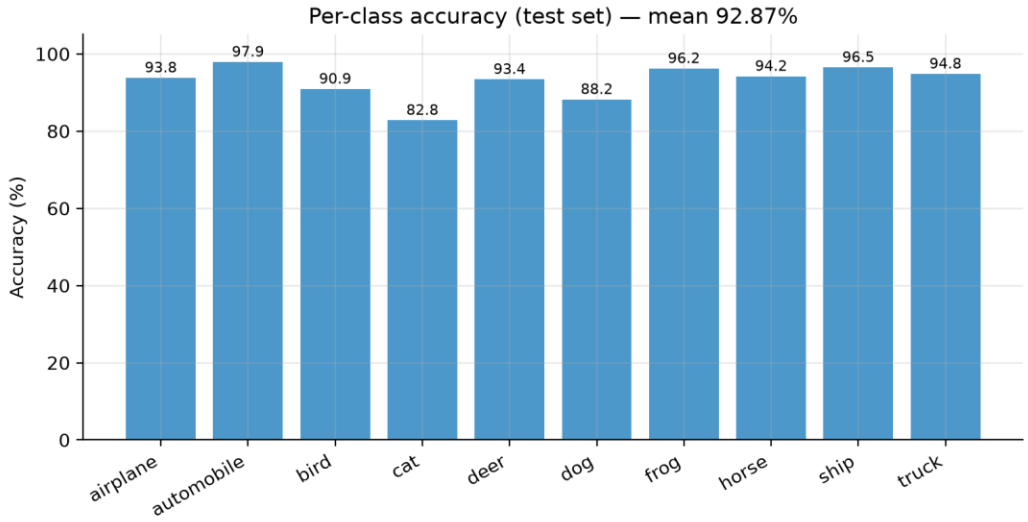


Figure 3. Per-class test accuracy (TTA). The macro average is 92.87 %.

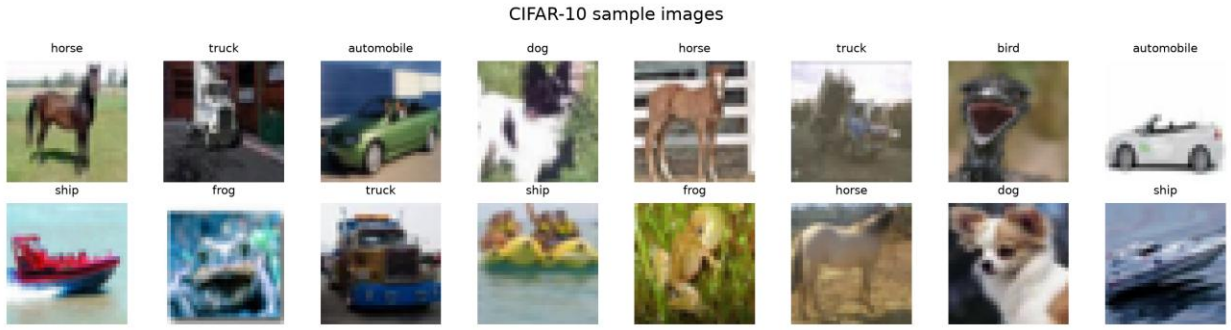


Figure 4. Sample images from the CIFAR-10 validation split with their labels, used to verify the download, the normalisation statistics and the augmentation pipeline.

4.6 Controlled comparison: the effect of numerical precision

The first full-scale attempt ran on a free GPU that supports only fp16, not bfloat16. It finished all 60 epochs without a single error message, yet the test accuracy was 25.73 % and the training loss never dropped below 2.15. Because the run's artefacts were complete, the failure could be pinned down exactly: the gradient scaler's dynamic loss scale had collapsed to 0.0156 (the default is 65536), and the Adam second-moment buffers of the first two stages were exactly zero, meaning those weights had never received a gradient. The cause is that layer scale initialises the residual branches with a factor of 1e-6, so the gradients in the shallow blocks are of order 1e-6; once the loss scale has collapsed, multiplying them by 0.0156 yields values below the smallest representable fp16 magnitude, and they are flushed to zero. The effective network then shrinks to the stem, the normalisation layer and the classifier head, a shallow linear model, which explains the 25 % plateau. Re-running the identical recipe with bfloat16, whose exponent range matches fp32, removes the problem entirely.

Table 12. The same recipe under two numerical precisions.

Aspect	fp16 on a free GPU (failed run)	bfloat16 on the local GPU (main result)
Gradient-scaler loss scale	collapsed to 0.0156	grew from 6.55e4 to 6.71e7

Aspect	fp16 on a free GPU (failed run)	bfloat16 on the local GPU (main result)
Shallow-block parameters	second-moment buffers exactly zero — never updated	updated normally
Training loss over 60 epochs	2.15 – 2.27 (ln 10 = 2.303)	2.03 → 0.87
Best validation accuracy	26.66 % (epoch 2)	93.50 % (epoch 59)
Test Top-1	25.73 % (invalid)	92.66 % (92.87 % with TTA)

The comparison is also a methodological lesson: a run that ends without errors is not automatically a valid run. The project now records the loss scale every epoch, stores the precision metadata inside the results files and prints a clear warning whenever the scale collapses. The code goes one step further and switches to full precision automatically when that happens, and the FP32 validation run (Table 13) confirms that the recipe itself is sound.

4.7 CPU baseline for pipeline validation

Table 13. Small-scale CPU run used to validate the complete pipeline before scaling up.

Item	Value
Experiment tag	mvp_atto_subset5k
Model / data	ConvNeXt-atto (3.38 M parameters) / 4,500 training, 500 validation, 2,000 test images
Training	AdamW, lr 2e-3, 10 epochs, batch 128, no Mixup, CPU (20 logical cores), 18 min 41 s
Result	Test Top-1 45.05 %, Top-5 89.95 %, best validation 45.20 %
Purpose	Confirms that data loading, augmentation, optimisation, evaluation and plotting all work on real data before spending GPU time

4.8 Discussion and limitations

- Accuracy level. The test Top-1 of 92.66 % (92.87 % with TTA) falls at the lower end of the 93–95 % band that the design notes predicted for this recipe on CIFAR-10. The main reason is the limited training budget rather than the architecture: validation accuracy was still improving at epoch 59 (92.9 % at epoch 50 → 93.5 % at epoch 59), so the model had not saturated.
- Hardware-induced choices. With 8 GB of video memory the batch size is capped at 128, and larger batches usually give modernised ConvNets a small accuracy gain. The evaluation therefore reflects a single-seed, single-configuration result rather than a tuned optimum.
- Strengths observed. The model generalises well: the gap between validation (93.50 %) and test (92.66 %) is only 0.84 percentage points, and no overfitting appears across 60 epochs despite 27.8 M parameters.
- Suggested follow-ups, in decreasing order of expected benefit. (1) Train for longer, 120 to 300 epochs, since the curve had not saturated. (2) Repeat with three seeds and report the mean and standard deviation. (3) Add RandAugment, which the code already supports through a configuration switch. (4) Run the structural ablations the project is already set up for, such as patchify stem versus 3x3 stem and the atto, pico and tiny presets. (5) Compare against a transformer baseline under identical settings to test the paper's central claim on this dataset.

5. What I Have Learnt from This AI Assignment

5.1 An AI agent can own the entire pipeline, but verification remains the human contribution

The agent wrote the model, the training loop, the evaluation, the plotting code, the Colab notebook generator, the download accelerator and the documentation, roughly 2,600 lines across 24 files, and none of it was hand-written. What it could not do on its own was decide whether the artefacts it produced were trustworthy. My most valuable contributions were procedural: insisting that every run write its configuration and per-epoch metrics to disk, that all reported numbers come from the same evaluation protocol, and that no claim be accepted until a measurement backs it. The fp16 incident is the clearest example. The code was correct and the logs showed no error, yet the result was worthless, and only the archived checkpoint made that visible.

5.2 Numerical precision is an engineering problem, not a detail

I had assumed that mixed precision was purely a speed optimisation. In fact 16-bit arithmetic can quietly destroy a training run: because layer scale multiplies each residual branch by 1e-6, the shallow blocks have very small gradients, and once the gradient scaler's dynamic loss scale collapsed, those gradients underflowed to exactly zero. The lesson is general: whenever a network contains very small multiplicative constants, bfloat16 (or full precision) is the safe default, and the loss scale should be logged and watched rather than assumed to be healthy.

5.3 Platform behaviour belongs to the experiment

Two of the three problems were specific to Windows. The operating system lets GPU memory be over-committed, so a memory probe cannot be trusted unless it keeps a safety margin; and PyTorch's multi-process data loader slowly eats into the system commit charge until the process dies after a couple of hours. Neither is a bug in the algorithm, but both would have distorted the results had they gone unnoticed. The habits I took away are to record the environment explicitly in the report, to keep long runs resumable and to treat a sudden change in per-epoch time as something worth investigating.

5.4 Reproducibility is a design decision taken in advance

The features that made the diagnosis possible — a tag per run, a complete hyper-parameter snapshot, per-epoch history written after every epoch, checkpoints that carry the optimiser and scaler state, and metrics exported in a fixed column order — were all specified before the experiments ran. Adding them afterwards would have been impossible, because the evidence would already have been gone. I now treat this kind of instrumentation as a first-class part of an experiment rather than an optional extra.

5.5 How to prompt an AI agent effectively

- Convert the specification into verifiable constraints first. Asking for a checklist before asking for a solution kept non-compliant or infeasible algorithms out of the running.
- Ask for justification backed by evidence. Requiring paper identifiers, links and licences made the literature claims verifiable rather than merely plausible.

- Ask the AI to argue against its own proposal. That is how the small-resolution stem problem came up, and it turned out to be the most important architectural adaptation in the project.
- Ask for artefacts rather than opinions. A request like “write the config and the per-epoch metrics to disk” produces evidence; a request like “is it working?” produces prose.
- Let the AI analyse the artefacts it produced. Both decisive diagnoses in this project, the collapsed loss scale and the never-updated parameters, came from asking the agent to inspect its own checkpoints, which it did far faster and more systematically than I could have.

5.6 What I would do differently next time

- Log the loss scale and mixed-precision metadata from the very first run instead of adding them after a failure (they are now part of the code).
- Validate on a small real-data subset before launching any multi-hour job — the CPU MVP run cost 19 minutes and would have caught environment problems immediately.
- Prefer bfloat16-capable hardware, or simply disable mixed precision, when the network contains very small multiplicative constants.
- Plan the comparison experiments (baselines, ablations, multiple seeds) from the start, so that the report can discuss them rather than only report a single configuration.

6. Source Code Webpage Link

The complete source code of this project is published at:

https://github.com/umjohn123/convnext-cifar10

A note on this item. The assignment requires an accessible web link, and the report is the only item that is submitted: the code, the data description, the experimental results and the AI dialogue records are supporting material, and all of it is available through the link above. The repository was published from the project structure described in Section 6.1 without modification. The raw CIFAR-10 archive and the model checkpoints are excluded from it for size, and both can be regenerated with the commands in Section 6.2.

6.1 What the repository contains

- The complete PyTorch project (model, training, evaluation, utilities, downloader) plus the self-checking Colab notebook generator.
- The dataset description, the split index files and the download instructions (the 170 MB archive itself is not committed).
- Every experiment with its figures, logs, metrics and checkpoints, including the failed run and the diagnosis of it.
- This report, in Word and PDF format.
- The dialogue records and the algorithm profile that document how the AI was used.

These folders are supporting material: the report is the only item that is submitted, and everything listed here is reached through the repository link in Section 6.

6.2 How to run the code

```
# 1) environment (PyPI's Windows wheels are CPU-only; use the CUDA index for GPU work)
pip install --upgrade --extra-index-url https://download.pytorch.org/whl/cu130 \
            "torch==2.14.1+cu130" "torchvision==0.29.1+cu130"
pip install -r requirements.txt

# 2) full-scale training (ConvNeXt-Tiny, 60 epochs, bfloat16, one GPU)
python train.py --run-tag full_convnext_tiny_cifar10 --model tiny --epochs 60 \
    --batch-size 128 --stem-mode conv --device cuda --num-workers 0 \
    --data-root cifar10 --output-dir results

# 3) evaluation with test-time augmentation
python evaluate.py --ckpt best.pt \
    --device cuda --tta --data-root cifar10 --output-dir results
```

The repository also contains a self-contained Colab notebook (ConvNeXt_CIFAR10_Colab.ipynb) that rebuilds the whole project inside Colab, downloads CIFAR-10, runs the training and displays the figures, so the experiments can be reproduced with no local setup.

References

6. Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell and S. Xie, “A ConvNet for the 2020s”, in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022, pp. 11976–11986. arXiv:2201.03545.
7. Official ConvNeXt implementation, Facebook Research (MIT License): https://github.com/facebookresearch/ConvNeXt
8. A. Krizhevsky, “Learning multiple layers of features from tiny images”, technical report, University of Toronto, 2009 (CIFAR-10 dataset).
9. H. Zhang, M. Cisse, Y. N. Dauphin and D. Lopez-Paz, “mixup: Beyond empirical risk minimization”, ICLR 2018 (Mixup).
10. S. Yun, D. Han, S. Chun, S. Oh, Y. Yoo and J. Choe, “CutMix: Regularization strategy to train strong classifiers with localizable features”, ICCV 2019 (CutMix).
11. G. Huang, Y. Sun, Z. Liu, D. Sedra and K. Weinberger, “Deep networks with stochastic depth”, ECCV 2016 (DropPath).
12. I. Loshchilov and F. Hutter, “Decoupled weight decay regularization”, ICLR 2019 (AdamW).
13. PyTorch documentation — automatic mixed precision and CUDA memory management: https://docs.pytorch.org/docs/stable/notes/cuda.html